

PONDICHERRY UNIVERSITY



SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

Name : VIDYAVHANY P
Register Number : 23384128
Semester : VI
Subject : Network Programming
Subject code : BCMI356

TABLE OF CONTENTS

EX. NO.	TITLE
1	PROGRAM SINGLE THREADING IN A SOCKET LAYER USING PYTHON PROGRAMMING
2	PROGRAM MULTIPLE THREADING IN A SOCKET LAYER USING PYTHON PROGRAMMING
3	PROGRAM HOW TO CACHE MEMORY WORK USING PYTHON PROGRAMMING
4	PROGRAM HOW TO HASHING AND SHADING WORK USING PYTHON PROGRAMMING
5	PROGRAM TO IMPLEMENT HTTP USING THE REQUESTS LIBRARY USING PYTHON PROGRAMMING

AIM: To develop a single-threaded client-server communication using Python sockets, where the server can handle only one client at a time.

PROCEDURE:

1. Import the socket module in both server and client programs.
2. Server Side:
 - Create a socket using `socket.socket()`.
 - Bind the socket to a specific IP address (localhost) and port (5000).
 - Put the server in listening mode using `listen()`.
 - Accept the incoming connection using `accept()`.
 - Receive message from client using `recv()`.
 - Send response to client using `send()`.
 - Repeat communication until termination.
 - Close the connection.
3. Client Side:
 - Create a socket using `socket.socket()`.
 - Connect to the server using `connect()`.
 - Take user input.
 - Send message to server using `send()`.
 - Receive response using `recv()`.
 - Continue until user stops the program.
4. Run the server first, then run the client.

CODE:

SERVER.PY

```
import socket
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
server.bind(("localhost", 5000))
```

```
server.listen(1)
```

```
print("Server waiting for connection...")
```

```
conn, addr = server.accept()
```

```
print("Connected to:", addr)
```

```
while True:
```

```
    data = conn.recv(1024).decode()
```

```
    if not data:
```

```
        break
```

```
    print("Client:", data)
```

```
    msg = input("Enter message: ")
```

```
    conn.send(msg.encode())
```

```
conn.close()
```

```
server.close()
```

CLIENT.PY

```
import socket
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client.connect(("localhost", 5000))
```

```
while True:
```

```
    msg = input("Enter message: ")
```

```
    client.send(msg.encode())
```

```
    data = client.recv(1024).decode()
```

```
    print("Server:", data)
```

OUTPUT:

<pre>Active code page: 65001 F:\DS LAB\SIMPLE_COMPOUND_INTRSET>CD F:\NP LAB F:\NP LAB>python server.py Server waiting for connection... Connected to: ('127.0.0.1', 62270) Client: hii Enter message: hello Client: i am vidya Enter message: vhany</pre>	<pre>Active code page: 65001 F:\DS LAB\SIMPLE_COMPOUND_INTRSET>CD F:\NP LAB F:\NP LAB>python client.py Enter message: hii Server: hello Enter message: i am vidya Server: vhany Enter message: </pre>
---	---

RESULT:

The Single threading in a socket layer using python program was executed successfully.

AIM: To implement a multi-threaded client-server communication using Python sockets, where the server can handle multiple clients simultaneously using threads.

PROCEDURE:

1. Import socket and threading modules.
2. Server Side:
 - Create a socket using `socket.socket()`.
 - Bind it to IP (localhost) and port (5001).
 - Enable listening using `listen()`.
 - Continuously accept multiple client connections using `accept()`.
 - For each client:
 - Create a new thread using `threading.Thread()`.
 - Call a function (`handle_client`) to manage communication.
 - Inside the thread:
 - Receive data from client using `recv()`.
 - Send response using `send()`.
 - Close connection when done.
3. Client Side:
 - Create a socket using `socket.socket()`.
 - Connect to server using `connect()`.
 - Send user input messages.
 - Receive and display server responses.
4. Run the server and then run multiple clients to test parallel communication.

CODE:

SERVER.PY

```
import socket  
  
import threading
```

```

def handle_client(conn, addr):
    print("Connected to:", addr)
    while True:
        try:
            data = conn.recv(1024).decode()
            if not data:
                break
            print(f'Client {addr}: {data}')
            msg = input(f'Reply to {addr}: ')
            conn.send(msg.encode())
        except:
            break
    conn.close()
    print("Connection closed:", addr)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 5001))
server.listen(5)
print("Server waiting for multiple clients...")
while True:
    conn, addr = server.accept()
    t = threading.Thread(target=handle_client, args=(conn, addr))
    t.start()

```

CLIENT.PY

```

import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 5001))

while True:
    msg = input("Enter message: ")
    client.send(msg.encode())
    data = client.recv(1024).decode()

```

```
print("Server:", data)
```

OUTPUT:

<pre>F:\DS LAB\SIMPLE_COMPOUND_INTRSET>cd F:\DS LAB\SIMPLE_COMPOUND_INTRSET F:\DS LAB\SIMPLE_COMPOUND_INTRSET>cd F:\WP LAB F:\WP LAB>python Multiserver.py Server waiting for multiple clients... Connected to: ('127.0.0.1', 52503) Connected to: ('127.0.0.1', 52504) Client ('127.0.0.1', 52503): hi Reply to ('127.0.0.1', 52503): Client ('127.0.0.1', 52504): ji </pre>	<pre>Active code page: 65001 F:\DS LAB\SIMPLE_COMPOUND_INTRSET>cd F:\DS LAB\SIMPLE_COMPOUND_INTRSET F:\DS LAB\SIMPLE_COMPOUND_INTRSET>cd F:\WP LAB F:\WP LAB>python Multiclient.py Enter message: hi </pre>	<pre>Active code page: 65001 F:\DS LAB\SIMPLE_COMPOUND_INTRSET>cd F:\WP LAB F:\WP LAB>python Multiclient.py Enter message: ji </pre>
--	---	--

RESULT:

The multiple threading in a socket layer using python program was executed successfully.

AIM: To implement cache memory (memoization) in Python to improve performance by storing previously computed results.

PROCEDURE:

1. Define a function to compute a value (e.g., factorial or Fibonacci).
2. Create a cache (dictionary) to store computed results.
3. Before computing, check if the value is already in cache.
4. If present, return it directly (cache hit).
5. Otherwise, compute the result and store it in cache (cache miss).
6. Display results and observe faster repeated calls.

CODE:

```
cache = {}  
  
def fibonacci(n):  
    if n in cache:  
        print("Cache hit for", n)  
        return cache[n]  
    print("Computing for", n)  
    if n <= 1:  
        result = n  
    else:  
        result = fibonacci(n - 1) + fibonacci(n - 2)  
    cache[n] = result  
    return result  
  
try:  
    num = int(input("Enter a number: "))  
    if num < 0:  
        print("Please enter a non-negative number.")  
    else:  
        result = fibonacci(num)
```

```
print("Fibonacci value is:", result)
except ValueError:
    print("Invalid input! Please enter a valid integer.")
```

OUTPUT:

```
Active code page: 65001
C:\Users\VIDYA\AppData\Local\Programs\Microsoft VS Code>C:\Users\VIDYA\AppData\Local\Programs\Python\Python313\python.exe "f:/NP LAB/Cache_Mem.py"
Enter a number: 1
Computing for 1
Fibonacci value is: 1
```

RESULT:

The cache memory work using python program was executed successfully.

AIM:To develop a Python program that demonstrates hashing and sharding techniques

PROCEDURE:

1. Start the program and display a menu with options:

- Hashing
- Sharding
- Exit

2. If the user selects Hashing:

- Create a dictionary to store key-value pairs.
- Accept input from the user.
- Generate a hash value using hash() function.
- Store the key-value pair in the dictionary.
- Allow the user to search for a key and display its value.

3.If the user selects Sharding:

- Create multiple dictionaries (shards).
- Use a hash function with modulo operation to select a shard.
- Store key-value pairs in the appropriate shard.
- Display all shard contents.
- Allow searching of data using the same hash logic.

4.If the user selects Exit:

- Terminate the program.

5.Ensure proper input handling and repeat the menu until exit

CODE:

```
def hashing():
```

```
    data = {}
```

```
    n = int(input("Enter number of elements: "))
```

```

for i in range(n):
    key = input("Enter key: ")
    value = input("Enter value: ")
    h = hash(key)
    print("Hash value of", key, "is:", h)
    data[key] = value # store in single dictionary
print("\nStored Data:", data)
search_key = input("Enter key to search: ")
if search_key in data:
    print("Value:", data[search_key])
else:
    print("Key not found")
def sharding():
    shards = [{} for _ in range(3)]
    def get_shard(key):
        return hash(key) % len(shards)
    n = int(input("Enter number of elements: "))
    for i in range(n):
        key = input("Enter key: ")
        value = input("Enter value: ")
        index = get_shard(key)
        print(f"Stored in Shard {index}")
        shards[index][key] = value
    print("\nShard Data:")
    for i in range(len(shards)):
        print(f"Shard {i}:", shards[i])
    search_key = input("Enter key to search: ")
    index = get_shard(search_key)
    print("Searching in Shard:", index)
    if search_key in shards[index]:

```

```
        print("Value:", shards[index][search_key])
    else:
        print("Key not found")
while True:
    print("\n--- MENU ---")
    print("1. Hashing")
    print("2. Sharding")
    print("3. Exit")
    choice = input("Enter your choice: ")
    if choice == '1':
        print("\n--- HASHING ---")
        hashing()
    elif choice == '2':
        print("\n--- SHARDING ---")
        sharding()
    elif choice == '3':
        print("Exiting program...")
        break
    else:
        print("Invalid choice! Try again.")
```

OUTPUT:

```
--- HASHING ---
Enter number of elements: 1
Enter key: 2
Enter value: 2
Hash value of 2 is: 4093865521294938808

Stored Data: {'2': '2'}
Enter key to search: 2
Value: 2

--- MENU ---
1. Hashing
2. Sharding
3. Exit
Enter your choice: 2

--- SHARDING ---
Enter number of elements: 1
Enter key: 1
Enter value: 1
Stored in Shard 2

Shard Data:
Shard 0: {}
Shard 1: {}
Shard 2: {'1': '1'}
Enter key to search: 1
Searching in Shard: 2
Value: 1
```

RESULT:

The Hashing and Sharding work using python program was executed successfully.

AIM: To implement HTTP communication using the requests library in Python for sending GET and POST requests to a web server.

PROCEDURE:

1. Install the requests library using:
 - pip install requests
2. Import the requests module in Python.
3. Send an HTTP GET request to retrieve data from a server.
4. Display the response status code and content.
5. Send an HTTP POST request with user data to the server.
6. Display the server response.

CODE:

```
import requests

url = input("Enter URL for GET request: ")

try:
    response = requests.get(url)
    print("\n--- GET REQUEST ---")
    print("Status Code:", response.status_code)
    print("Response Data:")
    print(response.text)
except Exception as e:
    print("Error:", e)

url = input("\nEnter URL for POST request: ")
data = {}
n = int(input("Enter number of data fields: "))
for i in range(n):
    key = input("Enter field name: ")
    value = input("Enter value: ")
    data[key] = value
```

```
response = requests.post(url, data=data)
print("\n--- POST REQUEST ---")
print("Status Code:", response.status_code)
print("Response Data:")
print(response.text)
```

```
print("Error:", e)
```

[illegible]

The http using the requests library using python program was executed successfully.